

16) Write an Embedded C program for UART communication with timeout handling and error management.

1. Problem Statement

The program implements UART communication using the AT89C51 microcontroller. It receives and transmits serial data and detects a timeout when data is not received within the specified period..

2. Aim

To design and implement an Embedded C program for UART serial communication using the AT89C51 microcontroller, with proper timeout detection and error handling.

3. Objectives

- Configure UART communication at 9600 baud.
- Transmit and receive serial data.
- Detect a timeout when no data is received.
- Handle communication errors safely.
- Verify the program using Keil μ Vision simulation.
-

4. Software and Hardware Requirements

Component	Specification
Microcontroller	AT89C51
Architecture	8051
Programming Language	Embedded C
Development Software	Keil μ Vision
Communication Protocol	UART
Baud Rate	9600 bps
Crystal Frequency	11.0592 MHz

5. Theory

UART (Universal Asynchronous Receiver/Transmitter) is a serial communication protocol used for communication between microcontrollers and other devices. It mainly uses TX for transmission and RX for reception.

In this program, UART is configured at 9600 baud. A timeout counter is used while waiting for incoming data. If no data is received before the timeout limit, the program reports a communication error instead of waiting indefinitely.

6. Algorithm

- 1. Start the program and initialize UART.**
- 2. Configure UART for 9600 baud.**
- 3. Wait for incoming UART data.**
- 4. Increment the timeout counter while waiting.**
- 5. If data is received, read and transmit the character back.**
- 6. Display SUCCESS for successful communication.**
- 7. If the timeout limit is reached, display a Receive Timeout error.**
- 8. Complete the program.**

7. Program

```
1  #include <reg51.h>
2
3  #define UART_OK 0
4  #define UART_TIMEOUT 1
5  #define TIMEOUT_LIMIT 50000UL
6
7  void UART_Init(void)
8  {
9      TMOD = 0x20;
10     TH1 = 0xFD;
11     TL1 = 0xFD;
12     SCON = 0x50;
13     TR1 = 1;
14 }
15
16 void UART_Transmit(char tx_char)
17 {
18     SBUF = tx_char;
19
20     while (TI == 0);
21
22     TI = 0;
23 }
24
25 void UART_SendString(char *str)
26 {
27     while (*str != '\0')
```

```
28 {
29     UART_Transmit(*str);
30     str++;
31 }
32 }
33
34 unsigned char UART_Receive(char *rx_char)
35 {
36     unsigned long timeout_count = 0;
37
38     while (RI == 0)
39     {
40         timeout_count++;
41
42         if (timeout_count >= TIMEOUT_LIMIT)
43         {
44             return UART_TIMEOUT;
45         }
46     }
47
48     *rx_char = SBUF;
49     RI = 0;
50
51     return UART_OK;
52 }
53
54 void main(void)
```

```

55 {
56     char received_char;
57     unsigned char status;
58
59     UART_Init();
60
61     UART_SendString("\r\n===== \r\n");
62     UART_SendString(" UART COMMUNICATION SYSTEM\r\n");
63     UART_SendString("===== \r\n");
64     UART_SendString("Baud Rate : 9600\r\n");
65     UART_SendString("Waiting for UART data...\r\n");
66
67     status = UART_Receive(&received_char);
68
69     if (status == UART_OK)
70     {
71         UART_SendString("\r\n--- UART RESULT ---\r\n");
72         UART_SendString("Data Received : ");
73         UART_Transmit(received_char);
74         UART_SendString("\r\nStatus : SUCCESS\r\n");
75         UART_SendString("No Error Detected\r\n");
76     }
77     else
78     {
79         UART_SendString("\r\n--- UART ERROR ---\r\n");
80         UART_SendString("ERROR : Receive Timeout\r\n");
81         UART_SendString("Status : Communication Failed\r\n");
82         UART_SendString("Action : Timeout handled safely\r\n");
83     }
84
85     UART_SendString("\r\nProgram Completed.\r\n");
86
87     while (1);
88 }

```

8. Program Explanation

- The UART_Init() function initializes Timer 1 and configures the UART for serial communication. The values TH1 = 0xFD and TL1 = 0xFD are used to obtain a baud rate of approximately 9600 bps with an 11.0592 MHz clock under the standard 8051 configuration.
- The UART_Transmit() function places a character in the SBUF register and waits for the transmission-complete flag TI.
- The UART_SendString() function repeatedly calls the transmit function to send an entire string.
- The UART_Receive() function waits for the receive flag RI. While waiting, it increments a timeout counter. If the timeout limit is reached before data arrives, UART_TIMEOUT is returned.
- The main() function initializes UART and tests the received status. Successful reception produces a SUCCESS message, whereas failure to receive data within the specified period generates a Receive Timeout error.

9. Test Case 1 – Successful UART Communication

Input

A character is entered through the Keil UART #1 simulation window.

Example:

A

Expected Output :

```
=====
      UART COMMUNICATION SYSTEM
=====
Baud Rate : 9600
Waiting for UART data...

--- UART RESULT ---
Data Received : A
Status       : SUCCESS
No Error Detected

Program Completed.
```

Actual Output:

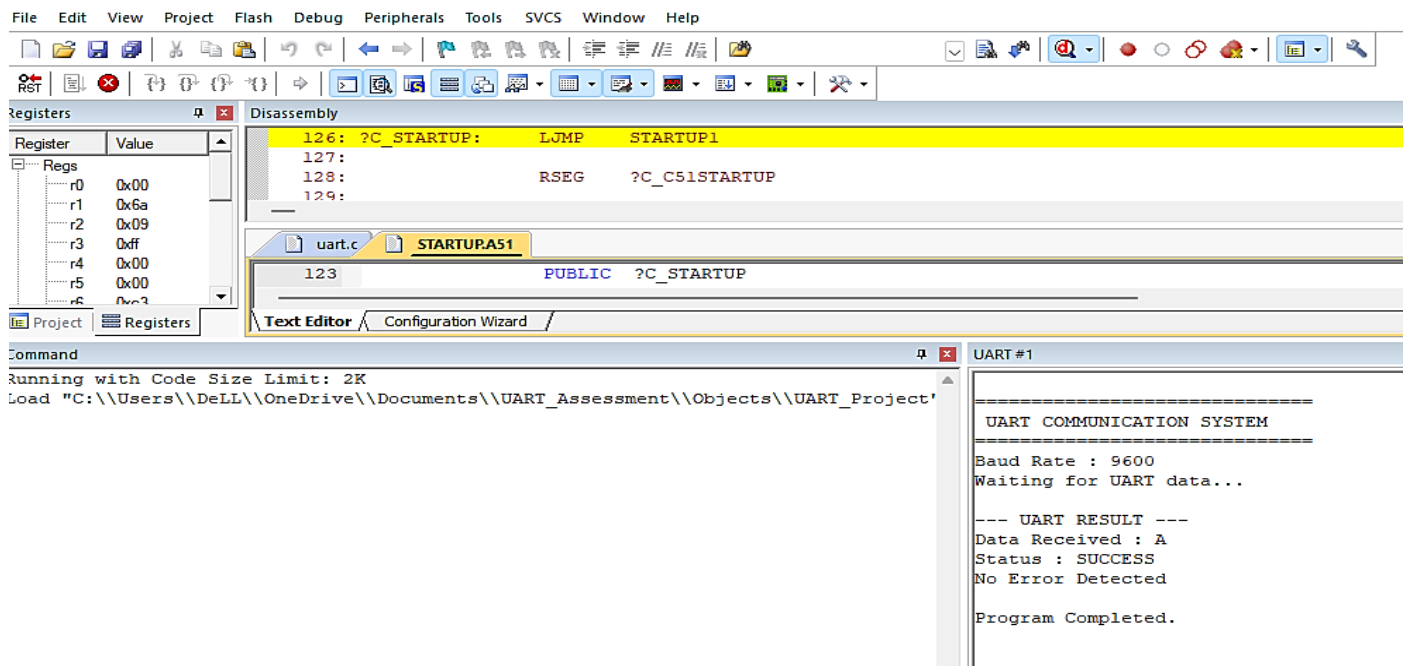


Figure 1: Successful UART data reception and transmission

Observation

The character 'A' was successfully received through UART. The received character was transmitted back, and the system displayed the SUCCESS status. Therefore, normal UART communication was verified successfully.

10. Test Case 2 – UART Timeout and Error Handling

Input

No character is provided to the UART receiver within the specified timeout period.

Expected Output

```
=====
UART COMMUNICATION SYSTEM
=====
Baud Rate : 9600
Waiting for UART data...

--- UART ERROR ---
ERROR : Receive Timeout
Status : Communication Failed
Action : Timeout handled safely

Program Completed.
```

Actual Output

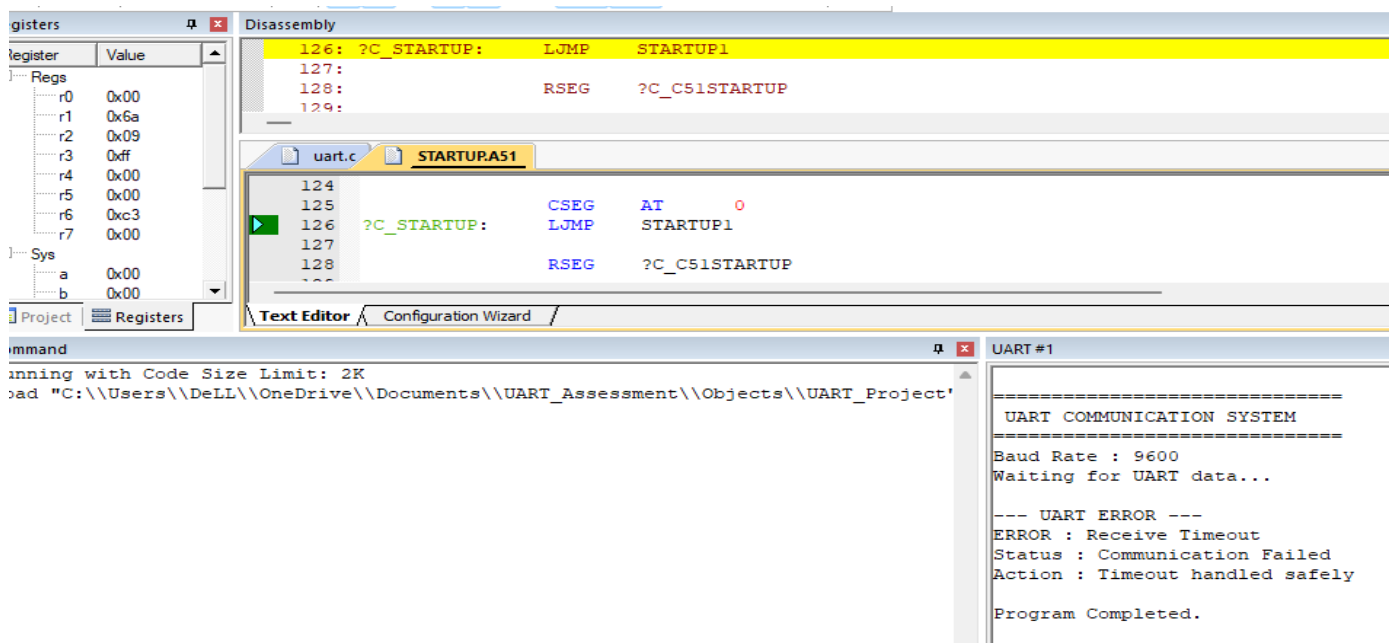


Figure 2: UART receive timeout and error handling

Observation

When no data was received within the specified period, the program detected the timeout condition. Instead of waiting indefinitely, it displayed a Receive Timeout error and safely terminated the communication attempt.

11. Test Case Comparison

Test Case	Input Condition	UART Result	Error Status	Final Result
Successful Communication	Character A received	Character received and echoed	No error	SUCCESS
Timeout Condition	No input	No character received	Receive timeout	HANDLED

The two test cases verify that the program can handle both normal communication and communication failure

13. Applications

- UART communication with timeout and error handling can be used in:
- Microcontroller-to-computer communication
- GPS modules
- Bluetooth modules
- GSM modules
- Sensor interfaces
- Industrial embedded systems
- IoT devices
- Automotive electronic systems

14. Result

1. The Embedded C program for UART communication with timeout handling and error management was successfully designed, compiled and simulated using the AT89C51 microcontroller in Keil μ Vision.
2. The program successfully received and transmitted UART data under normal operating conditions. It also detected the absence of incoming data using a timeout mechanism and handled the error safely.=

3. Hence, UART communication, timeout detection and error management were successfully implemented and verified.

15. Conclusion

This implementation demonstrates reliable UART communication in an embedded system. The AT89C51 microcontroller was configured to communicate at 9600 baud, and the program was tested for both successful reception and timeout conditions.

The timeout mechanism prevents indefinite blocking when data is unavailable, while the error-handling mechanism provides a clear indication of communication failure. Therefore, the developed program provides a simple and effective implementation of UART communication with timeout and error management in Embedded C.

19) Develop an Embedded C program implementing FSM-based control for an industrial process.

1. Problem Statement

The program implements a Finite State Machine (FSM) using the AT89C51 microcontroller to control an industrial process through four states: IDLE, STARTING, RUNNING, and STOPPED.

2. Aim

To design and implement an FSM-based industrial process controller using Embedded C and simulate its operation using Keil μ Vision.

3. Objectives

- Implement an industrial process using FSM.
- Define different operating states of the system.
- Perform controlled transitions between states.
- Control the motor/process according to the current state.
- Display the state information using UART.
- Verify the program using Keil μ Vision.

4. Software and Hardware Requirements

Component	Specification
Microcontroller	AT89C51
Programming Language	Embedded C
Software	Keil μVision
Compiler	Keil C51
Control Method	Finite State Machine
Output Port	Port 1 (P1)
Serial Output	UART
Baud Rate	9600 bps

5. Theory

A Finite State Machine (FSM) is a control technique in which a system operates in a fixed number of states. The system changes from one state to another according to predefined conditions.

In this program, an industrial process is represented using four states:

IDLE → STARTING → RUNNING → STOPPED

Each state performs a specific operation. This makes the industrial control process organized, predictable, and easy to manage.

6. Algorithm

- Start the program and initialize UART.
- Set the initial state as IDLE.
- In IDLE state, keep the motor OFF.
- Change to STARTING state and initialize the process.
- Change to RUNNING state and turn the motor/process ON.
- Change to STOPPED state after completing the process.
- Turn the motor OFF.
- Display each state through UART.
- Display FSM Execution Successful.

7. Embedded C Program

```
1 #include <reg51.h>
2
3 /* FSM State Definitions */
4 #define IDLE      0
5 #define STARTING  1
6 #define RUNNING   2
7 #define STOPPED   3
8
9 unsigned char current_state;
10
11 /* Initialize UART at 9600 baud */
12 void UART_Init(void)
13 {
14     TMOD = 0x20;
15     TH1 = 0xFD;
16     TL1 = 0xFD;
17     SCON = 0x50;
18     TR1 = 1;
19 }
20
21 /* Transmit one character */
22 void UART_Transmit(char tx_char)
23 {
24     SBUF = tx_char;
25
26     while (TI == 0);
27
28     TI = 0;
29 }
30
```

```
31 /* Transmit a string */
32 void UART_SendString(char *str)
33 {
34     while (*str != '\0')
35     {
36         UART_Transmit(*str);
37         str++;
38     }
39 }
40
41 /* Industrial Process FSM */
42 void FSM_Control(void)
43 {
44     switch (current_state)
45     {
46         case IDLE:
47
48             P1 = 0x00;
49
50             UART_SendString("\r\nState 1 : IDLE\r\n");
51             UART_SendString("Motor   : OFF\r\n");
52             UART_SendString("Process : Waiting\r\n");
53
54             current_state = STARTING;
55             break;
56
57         case STARTING:
58
59             P1 = 0x01;
60
61             UART_SendString("\r\nState 2 : STARTING\r\n");
62             UART_SendString("Motor   : ON\r\n");
63             UART_SendString("Process : Running\r\n");
64
65             current_state = RUNNING;
66             break;
67
68         case RUNNING:
69
70             P1 = 0x02;
71
72             UART_SendString("\r\nState 3 : RUNNING\r\n");
73             UART_SendString("Motor   : ON\r\n");
74             UART_SendString("Process : Running\r\n");
75
76             current_state = STOPPED;
77             break;
78
79         case STOPPED:
80
81             P1 = 0x03;
82
83             UART_SendString("\r\nState 4 : STOPPED\r\n");
84             UART_SendString("Motor   : OFF\r\n");
85             UART_SendString("Process : Waiting\r\n");
86
87             current_state = IDLE;
88             break;
89     }
90 }
```

```

61
62     UART_SendString("\r\nState 2 : STARTING\r\n");
63     UART_SendString("Motor   : STARTING\r\n");
64     UART_SendString("Process : Initializing\r\n");
65
66     current_state = RUNNING;
67     break;
68
69
70     case RUNNING:
71
72         P1 = 0x03;
73
74         UART_SendString("\r\nState 3 : RUNNING\r\n");
75         UART_SendString("Motor   : ON\r\n");
76         UART_SendString("Process : ACTIVE\r\n");
77
78         current_state = STOPPED;
79         break;
80
81
82     case STOPPED:
83
84         P1 = 0x00;
85
86         UART_SendString("\r\nState 4 : STOPPED\r\n");
87         UART_SendString("Motor   : OFF\r\n");
88         UART_SendString("Process : COMPLETED\r\n");
89
90         break;
91
92
93     default:
94
95         P1 = 0x00;
96         current_state = IDLE;
97         break;
98 }
99 }
100
101 void main(void)
102 {
103     UART_Init();
104
105     current_state = IDLE;
106
107     UART_SendString("\r\n===== \r\n");
108     UART_SendString("  INDUSTRIAL PROCESS FSM\r\n");
109     UART_SendString("===== \r\n");
110
111     FSM_Control();
112     FSM_Control();
113     FSM_Control();
114     FSM_Control();
115
116     UART_SendString("\r\n===== \r\n");
117     UART_SendString("  FSM EXECUTION SUCCESSFUL\r\n");
118     UART_SendString("===== \r\n");
119
120     while (1);
121 }

```

8. Program Explanation

- The program uses a Finite State Machine to control an industrial process. The initial state is IDLE. The system then moves sequentially through STARTING, RUNNING, and STOPPED.
- The FSM_Control() function uses a switch statement to perform the required operation for each state. Port 1 represents the industrial control output, while UART is used to display the current motor and process status.

9. Output

Expected Output

```
=====
  INDUSTRIAL PROCESS FSM
=====

State 1 : IDLE
Motor   : OFF
Process : Waiting

State 2 : STARTING
Motor   : STARTING
Process : Initializing

State 3 : RUNNING
Motor   : ON
Process : ACTIVE

State 4 : STOPPED
Motor   : OFF
Process : COMPLETED

=====
  FSM EXECUTION SUCCESSFUL
=====
```

Actual Output Screenshot

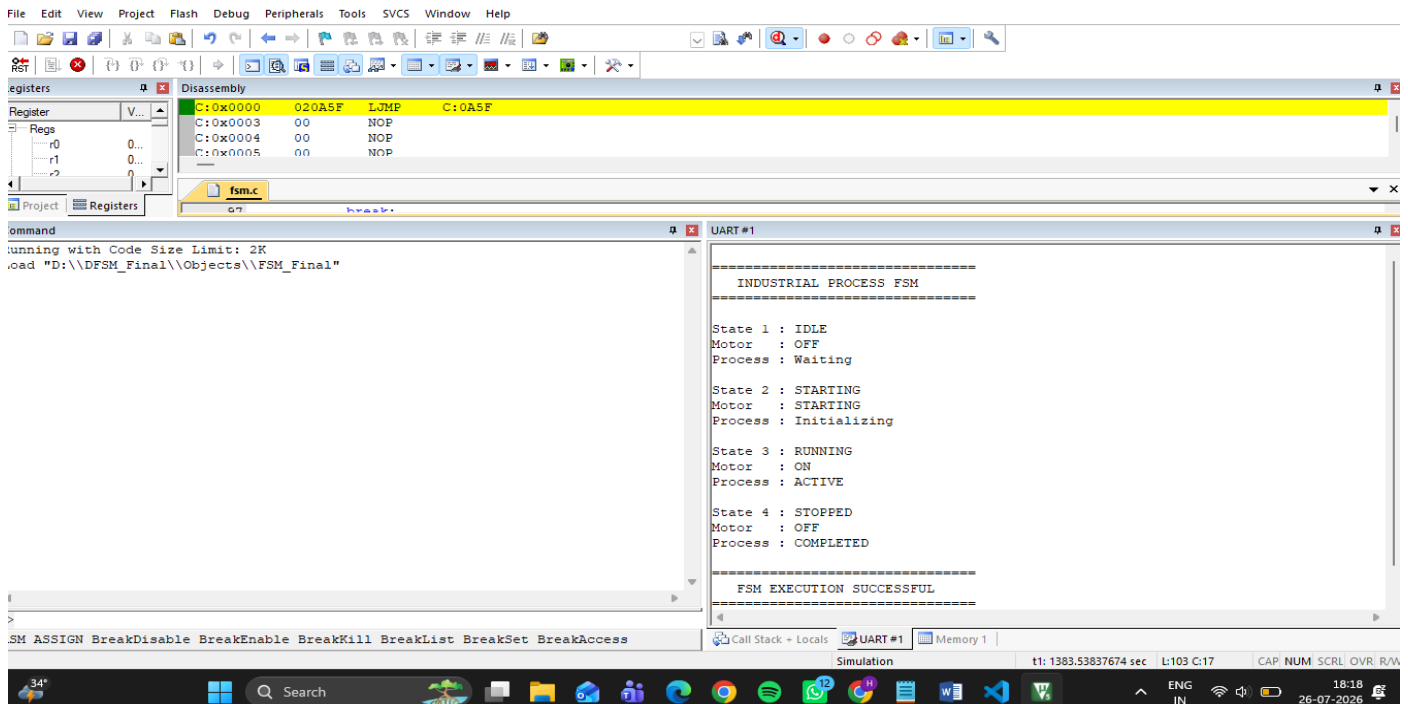


Figure 1: FSM-based industrial process control output

Observation

The industrial process successfully transitioned through all four states:

IDLE → STARTING → RUNNING → STOPPED

The motor and process status changed according to each state, and the FSM execution was completed successfully.

10. State Analysis

FSM State	Motor Status	Process Status
IDLE	OFF	Waiting
STARTING	STARTING	Initializing
RUNNING	ON	Active

FSM State	Motor Status	Process Status
STOPPED	OFF	Completed

11. Applications

- Industrial conveyor systems
- Motor control systems
- Automated manufacturing
- Packaging systems
- Process control systems
- Robotic control systems

12. Result and Conclusion

The FSM-based industrial process controller was successfully implemented using Embedded C and simulated with the AT89C51 in Keil μ Vision.

The system successfully transitioned through IDLE, STARTING, RUNNING, and STOPPED states. The corresponding motor and process conditions were displayed through UART. Hence, the FSM-based control of an industrial process was successfully implemented and verified.